The Simplex-2 Multiset Embedder and a Proof of Its Injectivity

Strategy, Implementation, and the Injectivity of Local Canonicalisation

June 21, 2026

Contents

1 What We Are Trying to Do

The input is an unordered multiset $M = \{\mathbf{v}_0, \ldots, \mathbf{v}_{n-1}\}$ of n vectors in $\mathbb{R}^k$. Concretely we represent M as a $k \times n$ matrix whose columns are the vectors $\mathbf{v}_j$. Column order is arbitrary.

We want a map f from such multisets to some $\mathbb{R}^N$ that is:

- **Multiset-invariant** — the output does not depend on how the columns are ordered.
- **Injective** — distinct multisets give distinct outputs (f is an *embedder*).
- **Continuous** (C^0) — small changes in the vectors produce small changes in $f(M)$.
- **Positively homogeneous of degree 1** — $f(\lambda M) = \lambda f(M)$ for all $\lambda > 0$.

Multiset invariance and continuity are in tension: any map that achieves invariance by sorting will have kinks wherever the sorted order changes. The strategy below resolves this tension elegantly.

2 The Strategy

Step A — Write M in a natural basis

Label each of the nk matrix entries by e_i^j (column j, row i). Let $X[j][i]$ denote the entry in column j, row i. Then

$$M \;=\; \sum_{j,i} X[j][i]\; e_i^j,$$

There is no harm in viewing the e_i^j as totally abstract/opaque basis vectors with no physical intepretation, but sometimes it can be helpful to view e_i^j concretely as the $k \times n$ matrix that is 1 at (i, j) and 0 elsewhere. This is the expansion of M in the *standard Eji basis*.

Step B — Peel off the fully symmetric parts

A *fully symmetric* basis element at row i is $\sum_j e_i^j$: it places the same coefficient on every vector at coordinate i, so it is unaffected by any permutation of the columns. The smallest coefficient that any vector has at row i is $m_i = \min_j X[j][i]$. Factoring:

$$M \;=\; \underbrace{\sum_i m_i \left(\sum_j e_i^j \right)}_{\text{symmetric part}} \;+\; \underbrace{\text{balance}}_{\text{all entries} \geq 0}\;.$$

The symmetric part is already permutation-invariant; its k scalar coefficients $m_0, \ldots, m_{k-1}$ can be stored directly as anchor values.

The balance $M - \sum_i m_i(\sum_j e_i^j)$ has all non-negative entries and contains the non-trivial, non-symmetric information in M.

Step C — Re-express the balance as a positive combination of nested basis elements

Within row i, order the n vectors descending by their (shifted) value. Let $\sigma_i(0), \ldots, \sigma_i(n-1)$ be this ordering. The gaps $\delta_r^{(i)} = X[\sigma_i(r)][i] - X[\sigma_i(r+1)][i] \geq 0$ are the drops between consecutive ranks.

By Abel summation (the discrete analogue of integration by parts; we also call this a *barycentric subdivision*, or BSD), the balance at row i becomes:

$$\text{balance at row } i \;=\; \sum_{r=0}^{n-2} \delta_r^{(i)} \cdot V_r^{(i)},$$

where the *prefix basis element* $V_r^{(i)} = \{e_i^{\sigma_i(0)}, \ldots, e_i^{\sigma_i(r)}\}$ is the set of the top-$(r{+}1)$ vectors by row-i value.

This is another change of basis: from individual Ejis with signed coefficients to nested prefix sets with non-negative gap coefficients. The structure of which vectors appear in $V_r^{(i)}$ — which j's dominate row i — is the non-symmetric information not yet discarded.

Step D — Merge, re-sort, and apply a second Abel summation

Collect all $m = k(n-1)$ gap–prefix-element pairs from all rows into one list and sort descending by gap: $\delta_{(0)} \geq \cdots \geq \delta_{(m-1)}$ with corresponding basis elements $V_{(0)}, \ldots, V_{(m-1)}$.

Apply a second Abel summation to obtain *cumulative* basis elements and weighted-difference coefficients:

$$\alpha_s \;=\; (s+1)\big(\delta_{(s)} - \delta_{(s+1)}\big), \qquad \delta_{(m)} := 0,$$

$$L_s \;=\; V_{(0)} + V_{(1)} + \cdots + V_{(s)},$$

where L_s is the *Eji_LinComb* (also called a *snapshot*; see §7): the $k \times n$ matrix whose (i, j) entry counts how many of the first $s+1$ prefix elements contain e_i^j. Its *index* $s+1$ records how many prefix elements have been accumulated.

The mathematical identity $\sum_s \delta_{(s)} V_{(s)} = \sum_s \alpha_s \cdot (L_s/(s+1))$ confirms this preserves all information. The normalised basis element $L_s/(s+1)$ is the *average* of the first $s+1$ prefix elements. The $(s+1)$ weight in α_s distributes contributions more evenly and ensures that $\sum_s \alpha_s = \sum_s \delta_{(s)}$ (the total weight is preserved).

Crucially, L_s captures *which prefix elements came first* in the global sort: two different orderings of the same set of prefix elements produce different L_s sequences.

Step E — Achieve permutation invariance by canonicalising

The j-indices in L_s still reflect the arbitrary labelling of columns. The *canonical form* $\widetilde{L_s}$ is obtained by sorting the columns of L_s's count matrix lexicographically — this removes the column labelling while retaining the multiset structure. Two inputs related by a column permutation produce identical $\widetilde{L_s}$ sequences and hence identical outputs. That each L_s is canonicalised *on its own*—a *local* column sort that discards how the columns of one L_s correspond to those of another—is what makes this step cheap and continuous; that it nonetheless cannot make two genuinely different multisets collide is the one substantive ingredient of injectivity, stated and proved in §7 (*Injectivity*).

Step F — Encode in $\mathbb{R}^{2m+1}$ and assemble

Map each canonical L_s to a pseudorandom point $p_s = h(\widetilde{L_s}) \in [0,1]^{2m+1}$ via MD5 hashing of its count matrix and index. The main body of the output is $\sum_s \alpha_s p_s$.

> **Why $\mathbb{R}^{2m+1}$ is the right dimension.** All $\alpha_s \geq 0$, so $\sum_s \alpha_s p_s$ is a point in the *positive cone* generated by $\{p_0, \ldots, p_{m-1}\}$, a cone of dimension m. Two m-dimensional positive cones in $\mathbb{R}^{2m+1}$ are *generically disjoint* by dimension counting: $m + m = 2m < 2m + 1$. Therefore a single point in $\mathbb{R}^{2m+1}$ encodes both: (a) which cone it lies in (i.e. which canonical $\{L_s\}$, i.e. which basis elements), and (b) the coefficients $\{\alpha_s\}$ within that cone.
> This is also why the algorithm is C^0: when two values in the same row are equal, the gap at that position is 0 and so $\alpha_s = 0$. The point $\sum \alpha_s p_s$ then lies on a shared *face* of two adjacent cones rather than in either interior. That shared face is the geometric expression of the continuity: the change in which cone we are in happens exactly at the moment when the coefficient of the transitioning basis element is zero, so the output is unaffected.

The full output is:

$$f(M) \;=\; \Big[\underbrace{m_0, \ldots, m_{k-1}}_{k \text{ row minima}},\; \underbrace{\textstyle\sum_{s=0}^{m-1} \alpha_s p_s}_{2m+1 \text{ reals}}\Big] \;\in\; \mathbb{R}^{k+2m+1}.$$

With $m = k(n-1)$: total size $= k + 2k(n-1) + 1 = 2nk + 1 - k$.

3 Two Algorithms, One Strategy

Both algorithms follow Steps A–F. They differ only in **Step B**: how many symmetric parts are peeled off, and what "fully symmetric" means.

Algorithm 2 (per-row). Factors out the k per-row symmetric elements $\sum_j e_i^j$, one per coordinate. Stores k row minima as anchors. Steps C–F operate on $m = k(n-1)$ gaps. Implemented as `Embedder.embed_generic` in `COHomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` (direct) and `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` (EncDec-backed); the `COHomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py` wrapper selects between them. See also `simplex_2_preprocess_steps` in `EncDec.py` and Appendix A.

Algorithm 1 (global). Factors out only the *single* fully-symmetric element $\sum_{i,j} e_i^j$ (treating every entry identically), with coefficient = global minimum. The global maximum is also recorded (see note below). Steps C–F then operate on a single globally-sorted list of $m = nk - 1$ gaps. Implemented as `Embedder.embed_generic` in `COHomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py` and as `simplex_1_preprocess_steps` in `EncDec.py`; see Appendix A.

The single global sort of Algorithm 1 means that prefix elements can contain Ejis from *any* row, whereas in Algorithm 2 every prefix element contains Ejis from exactly one row. This is why Algorithm 2 is conceptually cleaner: it respects the coordinate structure of the vectors.

Dimension count.

	Algorithm 1	**Algorithm 2**
Symmetric parts extracted	1 (global)	k (per-row)
Gaps m	$nk - 1$	$k(n-1) = nk - k$
Anchor values stored	2 (max & min)	k (row minima)
Hash embedding dimension $2m + 1$	$2nk - 1$	$2nk - 2k + 1$
Total output size	$\mathbf{2nk + 1}$	$\mathbf{2nk + 1 - k}$

Algorithm 2 saves k output dimensions because per-row symmetric extraction produces $k - 1$ fewer gaps than global extraction (since each row contributes $n-1$ gaps, totalling $k(n-1) = nk-k$, versus the global $nk - 1$ gaps from Algorithm 1), and the $(s + 1)$ weighting scheme in Step D ensures that the hash dimension savings exceed the extra anchor overhead.

Note on Algorithm 1's global maximum. In a perfect implementation, only the global minimum is needed as an anchor (it fixes the absolute scale; all relative gaps are encoded in the hash sum). The global maximum is technically redundant. It is included in Algorithm 1 for implementation simplicity; a TODO comment in the source code acknowledges this.

Source code.

Algorithm 2 — two interchangeable implementations (Steps A–F each):

- `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` — direct implementation; `Embedder.embed_generic`. This is the original file on which the document is based.
- `C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` — EncDec-backed; `Embedder.embed_generic`. Delegates Steps A–E to `EncDec.py` and adds Step F directly. Default config produces bit-identical output to `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`.
- `C0HomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py` — wrapper that selects between the above. All other code imports `Embedder` from this file.

Algorithm 1:

- `C0HomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py` — direct implementation; `Embedder.embed_generic`; Steps A–F.

Shared infrastructure:

- `EncDec.py` — `simplex_2_preprocess_steps` and `simplex_1_preprocess_steps` cover Steps A–E. Docstrings use this document's $\delta_{(s)}$, $V_{(s)}$, $\Delta_{(s)}$, $L_{(s)}$, α_s, $\hat{L}_{(s)}$ notation.
- `play_simplex_encoders_via_EncDec.py` — command-line driver for `EncDec.py`.

A parity test suite (`test_simplex2_ab_parity.py`) verifies that `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` and `C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` produce identical embeddings under the default configuration. See Appendix A for a full notation–code correspondence table.

4 Worked Example: Algorithm 2, $n = 4$, $k = 3$

All tables and numeric displays in this section are generated directly from `EncDec.py` by `DOCS/generate_simplex_example_tables.py` and `\input` from `DOCS/generated/`. To re-run the example with a different multiset, edit M in that script and rebuild (`make tables`).

$$M = \left\{ \begin{pmatrix} 4 \\ 2 \\ 3 \end{pmatrix}; \begin{pmatrix} -3 \\ -5 \\ 1 \end{pmatrix}; \begin{pmatrix} 8 \\ 9 \\ 2 \end{pmatrix}; \begin{pmatrix} 2 \\ -3 \\ -7 \end{pmatrix} \right\} \quad (n = 4 \text{ vectors in } \mathbb{R}^3)$$

Step B: Extract row minima

Row minima: $m_0 = -3$ (row 0), $m_1 = -5$ (row 1), $m_2 = -7$ (row 2). These three scalars are stored as anchors. After subtracting:

$$\text{balance} = \left\{ \begin{pmatrix} 7 \\ 7 \\ 10 \end{pmatrix}; \begin{pmatrix} 0 \\ 0 \\ 8 \end{pmatrix}; \begin{pmatrix} 11 \\ 14 \\ 9 \end{pmatrix}; \begin{pmatrix} 5 \\ 2 \\ 0 \end{pmatrix} \right\}$$

(All non-negative.) The debug check in the source code verifies that $\sum_s \alpha_s \cdot (L_s/(s+1)) + \text{broadcast}(m_i)$ recovers the original M exactly.

Step C: First Abel summation (once per row)

Each of the $k = 3$ coordinate rows is now fed through the Abel-summation operation — the *same* operation that Step D will apply once more to the merged results, and the same operation

that `EncDec.py` implements as `barycentric_subdivide`. Its input is a list of (coefficient, basis element) pairs; its output is a list of (gap, cumulative prefix) pairs plus one *offset* term. The three tables below (one per row) deliberately use the same layout as the large Step-D table: the left column pair is the input, the right column pair is the output.

Each row's input is its raw values $x^{(i)}_{(r)}$ paired with single Ejis; sorting descending and differencing gives the gaps $\delta^{(i)}_r = x^{(i)}_{(r)} - x^{(i)}_{(r+1)}$ and cumulative prefixes $V^{(i)}_r$. Note that the gaps are unchanged by the Step-B subtraction, and the offset term of each table — final value × all-ones row — is *precisely* the row-minimum anchor m_i of Step B. (In `EncDec.py`, Step B is not a separate pass: the anchors are simply the offset outputs of these three calls, requested via `return_offset_separately=True`.)

Row $i = 0$

			$x^{(0)}_{(r)} - x^{(0)}_{(r+1)}$	$V^{(0)}_{r-1} + e_0^{\sigma_0(r)}$
r	$x^{(0)}_{(r)}$	$e_0^{\sigma_0(r)}$	$\delta^{(0)}_r$	$V^{(0)}_r$
0	8	e_0^2	4	e_0^2
1	4	e_0^0	2	$e_0^0 + e_0^2$
2	2	e_0^3	5	$e_0^0 + e_0^2 + e_0^3$

offset $= x^{(0)}_{(3)} \cdot \sum_j e_0^j = -3\,(e_0^0 + e_0^1 + e_0^2 + e_0^3)$

Row $i = 1$

			$x^{(1)}_{(r)} - x^{(1)}_{(r+1)}$	$V^{(1)}_{r-1} + e_1^{\sigma_1(r)}$
r	$x^{(1)}_{(r)}$	$e_1^{\sigma_1(r)}$	$\delta^{(1)}_r$	$V^{(1)}_r$
0	9	e_1^2	7	e_1^2
1	2	e_1^0	5	$e_1^0 + e_1^2$
2	−3	e_1^3	2	$e_1^0 + e_1^2 + e_1^3$

offset $= x^{(1)}_{(3)} \cdot \sum_j e_1^j = -5\,(e_1^0 + e_1^1 + e_1^2 + e_1^3)$

Row $i = 2$

			$x^{(2)}_{(r)} - x^{(2)}_{(r+1)}$	$V^{(2)}_{r-1} + e_2^{\sigma_2(r)}$
r	$x^{(2)}_{(r)}$	$e_2^{\sigma_2(r)}$	$\delta^{(2)}_r$	$V^{(2)}_r$
0	3	e_2^0	1	e_2^0
1	2	e_2^2	1	$e_2^0 + e_2^2$
2	1	e_2^1	8	$e_2^0 + e_2^1 + e_2^2$

offset $= x^{(2)}_{(3)} \cdot \sum_j e_2^j = -7\,(e_2^0 + e_2^1 + e_2^2 + e_2^3)$

Normalisation (`preserve_scale`) is switched *off* at this layer in the standard configuration, so no $\alpha/\hat{V}$ column pair appears; the `COHomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py` implementation also permits it to be switched on here, which would add one, exactly as in Step D's table.

Step D: Merge, re-sort, and the same Abel summation again

The nine $(\delta_r^{(i)}, V_r^{(i)})$ output pairs of Step C now become the *input* pairs $(\delta_{(s)}, V_{(s)})$ of a second pass through the very same operation. Merge all $9 = k(n-1)$ pairs and sort by gap descending:

$$8,\ 7,\ 5,\ 5,\ 4,\ 2,\ 2,\ 1,\ 1$$

The table below has the same structure as Step C's tables — input pair on the left, output pair(s) on the right — but this time with normalisation switched *on*, which appends the $(\alpha_s, \hat{L}_{(s)})$ column pair to the unnormalised $(\Delta_{(s)}, L_{(s)})$ one. Terms within each L_s expression are grouped by coordinate row i.

			$\delta_{(s)} - \delta_{(s+1)}$	$L_{(s-1)} + V_{(s)}$	$(s+1)\Delta_{(s)}$	$L_{(s)}/(s+1)$
s	$\delta_{(s)}$	$V_{(s)}$	$\Delta_{(s)}$	$L_{(s)}$	α_s	$\hat{L}_{(s)}$
0	8	$e_2^0 + e_2^1 + e_2^2$	1	$e_2^0 + e_2^1 + e_2^2$	1	$e_2^0 + e_2^1 + e_2^2$
1	7	e_1^2	2	e_1^2 $+ e_2^0 + e_2^1 + e_2^2$	4	$\frac{1}{2}(e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
2	5	$e_0^0 + e_0^2 + e_0^3$	0	$e_0^0 + e_0^2 + e_0^3$ $+ e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	0	$\frac{1}{3}(e_0^0 + e_0^2 + e_0^3$ $+ e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
3	5	$e_1^0 + e_1^2$	1	$e_0^0 + e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	4	$\frac{1}{4}(e_0^0 + e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
4	4	e_0^2	2	$e_0^0 + 2e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	10	$\frac{1}{5}(e_0^0 + 2e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
5	2	$e_0^0 + e_0^2$	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	0	$\frac{1}{6}(2e_0^0 + 3e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
6	2	$e_1^0 + e_1^2 + e_1^3$	1	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2$	7	$\frac{1}{7}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2)$
7	1	e_2^0	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 2e_2^0 + e_2^1 + e_2^2$	0	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 2e_2^0 + e_2^1 + e_2^2)$
8	1	$e_2^0 + e_2^2$	1	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 3e_2^0 + e_2^1 + 2e_2^2$	9	$\frac{1}{9}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 3e_2^0 + e_2^1 + 2e_2^2)$
	$\sum_s \delta_{(s)} = 35$		$\sum_s \Delta_{(s)} = 8 \neq 35$		$\sum_s \alpha_{(s)} = 35$	
	$\sum_s \delta_{(s)} V_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$		$\sum_s \Delta_{(s)} L_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$		$\sum_s \alpha_{(s)} \hat{L}_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$	

Reading off an α_s. Each α_s is the product of the position weight $(s{+}1)$ and the gap difference $\Delta_{(s)} = \delta_{(s)} - \delta_{(s+1)}$, either of which can exceed 1. For example:

$$\alpha_1 \;=\; (1{+}1)\big(\delta_{(1)} - \delta_{(2)}\big) \;=\; 2 \times (7 - 5) \;=\; 4.$$

(The example multiset used in earlier versions of this document had, by coincidence, every non-zero gap difference equal to 1, making the non-zero α_s values misleadingly equal to $s{+}1$. The present multiset was chosen to avoid that.)

Verification: column sums and three-way equality. The integer columns $\delta_{(s)}$ and α_s have the same total:

$$\sum_{s=0}^{8} \delta_{(s)} \;=\; 8{+}7{+}5{+}5{+}4{+}2{+}2{+}1{+}1 \;=\; 35 \;=\; 1{+}4{+}0{+}4{+}10{+}0{+}7{+}0{+}9 \;=\; \sum_{s=0}^{8} \alpha_s.$$

The three weighted Eji sums are all equal and recover the `balance` matrix of Step B (the $L_{(s)}$ and $\hat{L}_{(s)}$ totals-row entries show the value directly; $\sum_s \delta_{(s)} V_{(s)}$ is identical):

$$\begin{aligned}\sum_s \delta_{(s)}\, V_{(s)} \;=\; \sum_s \Delta_{(s)}\, L_{(s)} \;=\; \sum_s \alpha_s\, \hat{L}_{(s)} = {} & 7e_0^0 + 11e_0^2 + 5e_0^3 \\ & + 7e_1^0 + 14e_1^2 + 2e_1^3 \\ & + 10e_2^0 + 8e_2^1 + 9e_2^2,\end{aligned}$$

i.e. coefficient matrix $\begin{pmatrix} 7 & 0 & 11 & 5 \\ 7 & 0 & 14 & 2 \\ 10 & 8 & 9 & 0 \end{pmatrix}$ (rows $= i = 0, \ldots, 2$; columns $= j = 0, \ldots, 3$), matching `balance` exactly.

In many implementations the basis elements $L_{(s)}$ or $\hat{L}_{(s)}$ are stored as matrices. For example, Eji_LinComb structures https://github.com/kesterlester/Echinocoder/blob/main/Eji_LinComb.py like those shown below (only for each term without a zero coefficient) are used in https://github.com/kesterlester/Echinocoder/blob/main/C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py. In these the count matrix has rows representing coordinates $i{=}0, 1, 2$; and columns representing vectors $j{=}0, 1, 2, 3$.

s	α_s	Eji_LinComb structure	$\hat{L}_{(s)} = L_{(s)}/(s{+}1)$
0	1	$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix}$, idx 1	$e_2^0 + e_2^1 + e_2^2$
1	4	$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix}$, idx 2	$\frac{1}{2}(e_1^2 + e_2^0 + e_2^1 + e_2^2)$
3	4	$\begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 2 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix}$, idx 4	$\frac{1}{4}(e_0^0 + e_0^2 + e_0^3 + e_1^0 + 2e_1^2 + e_2^0 + e_2^1 + e_2^2)$
4	10	$\begin{pmatrix} 1 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix}$, idx 5	$\frac{1}{5}(e_0^0 + 2e_0^2 + e_0^3 + e_1^0 + 2e_1^2 + e_2^0 + e_2^1 + e_2^2)$
6	7	$\begin{pmatrix} 2 & 0 & 3 & 1 \\ 2 & 0 & 3 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$, idx 7	$\frac{1}{7}(2e_0^0 + 3e_0^2 + e_0^3 + 2e_1^0 + 3e_1^2 + e_1^3 + e_2^0 + e_2^1 + e_2^2)$
8	9	$\begin{pmatrix} 2 & 0 & 3 & 1 \\ 2 & 0 & 3 & 1 \\ 3 & 1 & 2 & 0 \end{pmatrix}$, idx 9	$\frac{1}{9}(2e_0^0 + 3e_0^2 + e_0^3 + 2e_1^0 + 3e_1^2 + e_1^3 + 3e_2^0 + e_2^1 + 2e_2^2)$

(Terms with $\alpha_s = 0$ contribute nothing and are omitted.)

The table below repeats the construction with a different but equally valid tie-breaking order: within each group of equal gaps, the gap–basis-element assignment is rotated one place (for a two-element group, a swap). The tied groups here are $\delta = 5$ (at $s \in \{2, 3\}$), $\delta = 2$ (at $s \in \{5, 6\}$) and $\delta = 1$ (at $s \in \{7, 8\}$). The basis vectors in the greyed cells change; the non-greyed cells are identical to those in the table above. Critically the greyed cells all have basis-coefficients (scalars $\Delta_{(s)}$ or α_s) which are zero by construction ... and so the very places where the basis-vectors are ambiguous are also (by construction) the very places where those basis vectors do not need to be encoded. Equivalently the non-zero values of Δ_s or α_s all scale basis vectors that are unambiguously defined. This is the key mechanism by which the embedder is able to maintain continuity in its outputs despite having to use the outputs to both encode the coefficients (which change smoothly) and the basis vectors themselves (which change discontinuously at boundaries).

			$\delta_{(s)} - \delta_{(s+1)}$	$L_{(s-1)} + V_{(s)}$	$(s+1)\Delta_{(s)}$	$L_{(s)}/(s+1)$
s	$\delta_{(s)}$	$V_{(s)}$	$\Delta_{(s)}$	$L_{(s)}$	α_s	$\hat{L}_{(s)}$
0	8	$e_2^0 + e_2^1 + e_2^2$	1	$e_2^0 + e_2^1 + e_2^2$	1	$e_2^0 + e_2^1 + e_2^2$
1	7	e_1^2	2	e_1^2 $+ e_2^0 + e_2^1 + e_2^2$	4	$\frac{1}{2}(e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
2	5	$e_1^0 + e_1^2$	0	$e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	0	$\frac{1}{3}(e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
3	5	$e_0^0 + e_0^2 + e_0^3$	1	$e_0^0 + e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	4	$\frac{1}{4}(e_0^0 + e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
4	4	e_0^2	2	$e_0^0 + 2e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2$	10	$\frac{1}{5}(e_0^0 + 2e_0^2 + e_0^3$ $+ e_1^0 + 2e_1^2$ $+ e_2^0 + e_2^1 + e_2^2)$
5	2	$e_1^0 + e_1^2 + e_1^3$	0	$e_0^0 + 2e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2$	0	$\frac{1}{6}(e_0^0 + 2e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2)$
6	2	$e_0^0 + e_0^2$	1	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2$	7	$\frac{1}{7}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ e_2^0 + e_2^1 + e_2^2)$
7	1	$e_2^0 + e_2^2$	0	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 2e_2^0 + e_2^1 + 2e_2^2$	0	$\frac{1}{8}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 2e_2^0 + e_2^1 + 2e_2^2)$
8	1	e_2^0	1	$2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 3e_2^0 + e_2^1 + 2e_2^2$	9	$\frac{1}{9}(2e_0^0 + 3e_0^2 + e_0^3$ $+ 2e_1^0 + 3e_1^2 + e_1^3$ $+ 3e_2^0 + e_2^1 + 2e_2^2)$
	$\sum_s \delta_{(s)} = 35$		$\sum_s \Delta_{(s)} = 8 \neq 35$		$\sum_s \alpha_{(s)} = 35$	
	$\sum_s \delta_{(s)} V_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$		$\sum_s \Delta_{(s)} L_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$		$\sum_s \alpha_{(s)} \hat{L}_{(s)} =$ $7e_0^0 + 11e_0^2 + 5e_0^3$ $+ 7e_1^0 + 14e_1^2 + 2e_1^3$ $+ 10e_2^0 + 8e_2^1 + 9e_2^2$	

Step E: Canonicalise

Sort the columns of each count matrix lexicographically (the column-sort permutation "forgets" which vector was which, achieving multiset invariance). After canonicalisation, L_s becomes $\widetilde{L_s}$.

Step F: Hash and assemble

Each $\widetilde{L_s}$ (along with its index) is hashed to a point $p_s \in [0,1]^{19}$ (since $N_{\text{hash}} = 2{\times}9{+}1 = 19$). The output is:

$$f(M) \;=\; \big[\, \underbrace{-3,\ -5,\ -7}_{\text{3 row min.}},\ 1\,p_0 + 4\,p_1 + 4\,p_3 + 10\,p_4 + 7\,p_6 + 9\,p_8 \,\big] \;\in\; \mathbb{R}^{22}.$$

5 Worked Example: Algorithm 1, $n = 4$, $k = 2$

$$M \;=\; \left\{ \begin{pmatrix} 4 \\ 2 \end{pmatrix}, \begin{pmatrix} -3 \\ 5 \end{pmatrix}, \begin{pmatrix} 8 \\ 9 \end{pmatrix}, \begin{pmatrix} 2 \\ 7 \end{pmatrix} \right\} \quad (n = 4 \text{ vectors in } \mathbb{R}^2)$$

Step B: Extract global minimum (and maximum)

Global min $= -3$, global max $= 9$. Both stored as anchors. Balance has all non-negative entries.

Steps C–D: Global sort and Abel summation

The balance after subtracting the global minimum -3 from every entry is:

$$\left\{ \begin{pmatrix}7\\5\end{pmatrix}, \begin{pmatrix}0\\8\end{pmatrix}, \begin{pmatrix}11\\12\end{pmatrix}, \begin{pmatrix}5\\10\end{pmatrix} \right\}$$

Sorted globally descending (value: label):

$$12(e_1^2),\ 11(e_0^2),\ 10(e_1^3),\ 8(e_1^1),\ 7(e_0^0),\ 5(e_1^0),\ 5(e_0^3),\ 0(e_0^1).$$

The 7 gaps are $1, 1, 2, 1, 2, 0, 5$. After global re-sort by gap descending and second Abel summation: $\alpha_0 = 3,\ \alpha_2 = 3,\ \alpha_5 = 6$ (others zero).

$N_{\text{hash}} = 2(8-1)+1 = 15$. Output:

$$f(M) \ = \ \left[\, 3\,p_0 + 3\,p_2 + 6\,p_5,\ 9,\ -3\,\right] \ \in \ \mathbb{R}^{17},$$

numerically (from the code's debug output):

$[\,7.07,\ 6.91,\ 2.78,\ 3.50,\ 1.01,\ 5.51,\ 4.07,\ 4.90,\ 10.02,\ 9.51,\ 5.13,\ 2.97,\ 8.04,\ 5.34,\ 7.41,\ 9,\ -3\,]$.

6 Output Size and the Whitney Embedding Theorem

Both algorithms produce output of size $\approx 2nk$, which is not overhead to be engineered away — it is intrinsic. The input space $SP^n(\mathbb{R}^k)$ is an nk-dimensional manifold. The **Whitney Embedding Theorem** states that any smooth d-dimensional manifold embeds smoothly into $\mathbb{R}^{2d}$, and $2d$ is tight in general. With output $\approx 2nk$, both algorithms sit right at the Whitney bound.

The factor of ≈ 2 is in fact the price of the cone-dimension mechanism in Step F: encoding m positive coefficients together with the identities of m basis elements inside a single point requires $\mathbb{R}^{2m+1}$. This same mechanism is what delivers C^0 continuity (shared faces at zero-coefficient transitions). The factor of two is the minimum price of a smooth embedding, and the algorithms pay almost exactly that.

Whether $2nk$ (or $2nk+1$, or $2nk+1-k$) is optimal for the specific manifold $SP^n(\mathbb{R}^k)$, or whether the known structure of that space admits a tighter construction, remains open.

7 Injectivity: why local canonicalisation loses nothing

Section 1 asked for four properties. Steps A–F deliver multiset-invariance, C^0 continuity and degree-1 homogeneity by construction, and the discussion around Step D made the continuity mechanism explicit (the "greyed cells": basis vectors that are ambiguous at a tie are precisely the ones whose coefficient is zero there). *Injectivity* was the one desideratum left on a heuristic footing. This section discharges it—all but one elementary step—and the rest of this section supplies the proof.

What injectivity reduces to

The output (Step F) is assembled from the multiset of pairs $\left(\alpha_s,\ \widetilde{L_s}\right)$ together with the k anchors m_i. Grant for the moment the Step F cone mechanism: that $\sum_s \alpha_s\, h(\widetilde{L_s})$ recovers, generically, the multiset of (coefficient, basis-vector) pairs that built it—the "generically disjoint m-cones in $\mathbb{R}^{2m+1}$" argument of Step F, which we take here as elementary. End-to-end injectivity then rests on a single combinatorial question:

> Do the *locally* canonicalised basis vectors $\{\widetilde{L_s}\}$—each L_s column-sorted *on its own* in Step E—still determine the input multiset M up to the one global relabelling of its n columns that "multiset" allows?

The worry is real and invisible to numerical search. Step E sorts the columns of each L_s independently, throwing away which column of one L_s corresponds to which column of another. A priori, two genuinely different multisets might produce families $\{L_s\}$ that match *matrix-by-matrix* after independent sorting, yet are *not* related by any single, common column permutation. Such a collision would break injectivity, and—being a coincidence among sorted integer matrices—would live on a measure-zero set that random probing never hits.

The theorem, in the embedder's language

The proof that fills out the rest of this section rules that out. Restated in the notation of Part I:

Local canonicalisation is injective-safe. If, for two inputs, every locally canonicalised basis vector $\widetilde{L_s}$ on one side equals one on the other (Step E agreement), then a *single* column permutation σ carries the whole family $\{L_s\}$ of one input onto that of the other (*global* agreement). This is Theorem 7.1; the operative direction is

$$\text{local (per-matrix) agreement} \implies \text{one global } \sigma.$$

A single common σ on the n columns is exactly one relabelling of the n vectors—i.e. the two inputs are the *same multiset*. So Step E cannot merge distinct multisets.

The surprise is that the *weaker*, each-matrix-separately notion of agreement that Step E actually tests already *forces* the stronger, all-at-once one. That implication—local $\Rightarrow$ global—is precisely the direction injectivity needs, and it is the hard direction of Theorem 7.1 (the easy converse, global $\Rightarrow$ local, is not what we are after).

Dictionary

The proof is written self-containedly in combinatorial language; every object in it is one the embedder manufactures. The correspondence is exact:

Proof object (the part below)	**Embedder object (Part I)**	**What it is**
$\vec{v}_j \in s(n)$, leave-one-out order of n symbols	descending sort order of row i; left-out symbol = that row's arg min	per-row rank order
$coh(\vec{v}_j)_m$, indicator of prefix P_m	$V_r^{(i)}$, the top-$(r{+}1)$ set (output of Step C, 1st BSD)	nested prefix indicator
$U(\vec{v}_1, \ldots, \vec{v}_k)$, the $(n{-}1)k$ pieces	the $m = k(n{-}1)$ prefix basis elements pre-merge	the piece-set
ordering p of $\vec{U}$	the gap-descending sort of Step D	merge order
$C\vec{U}$, cumulative; $S^{\vec{v}}(Q)$, snapshot	$L_s = V_{(0)} + \cdots + V_{(s)}$, the Eji_LinComb count matrix (Step D, 2nd BSD)	cumulative basis matrix
position $= \sum_r \max_c(\cdot)_{rc}$	the stored index $s{+}1$ of L_s	step count
filter $\vec{f}$ then *clean*	dropping the $\alpha_s = 0$ terms	which snapshots survive
canon / *localSort* (Step E analogue)	Step E: sort each count matrix's columns on its own	local canonicalisation
globalSort	(not in the algorithm) one common σ for all L_s	multiset-faithful sort
$\Sigma(Q)$	column relabellings aligning the prefixes a snapshot reveals	alignment set

One notation seam, not yet sewn. Part I indexes the coordinate (row) by i and the vector (column) by j; the proof below indexes the coordinate (row) by j and treats the columns as the n *symbols*/positions. So the symbol j means a *row* in the proof and a *column* in Part I—read the dictionary, not the letters, until the two conventions are unified.

Why the proof carries a filter, two orderings and a "clean"

These three apparent complications in the statement of Theorem 7.1 are not decoration: each models exactly one feature of how the embedder behaves at ties, and discharging them is what the proof spends its effort on.

The filter. The cone sum sees only nonzero-coefficient terms, so a pair with $\alpha_s = 0$ is not dropped by choice—it is *invisible*. Which s survive is data-dependent (any pattern can occur, since $\alpha_s = (s{+}1)(\delta_{(s)} - \delta_{(s+1)})$ vanishes wherever two merged gaps coincide), so the safety statement must hold for *every* filter $\vec{f}$. Moreover $\alpha_s = 0$ holds exactly when L_s sits inside a run of equal gaps—exactly when L_s is a *tie-break-dependent* partial sum, the ambiguous basis vector of the "greyed cells". Hidden $\iff$ zero coefficient $\iff$ ambiguous: the very vectors continuity *forces* us to hide are the ones whose value we could not have pinned down anyway, and the survivors are the run-boundary sums, which are tie-break-invariant. The theorem's job is to prove this forced, data-dependent hiding is nonetheless lossless.

Two orderings p, q. Two nearby inputs straddling a tie break it in opposite ways—different orderings of a run's pieces. The proof allows $p \neq q$ and shows local agreement *forces them back into step*, so the arbitrary tie-break costs nothing.

The clean. The cone sum is both order-blind and zero-blind, so hidden positions leave no trace: the survivors arrive as a bare set with holes. The position-recovery lemma (Lemma 7.4; row-maxima sum = index) lets each survivor re-announce where it sat, refilling the holes—which is why *clean* may be undone and the families compared as plain sets.

Scope. This closes the *discrete* half of injectivity: the identities of the surviving basis vectors, and that local canonicalisation recovers them faithfully up to one global relabelling. The remaining half— that the Step F cone sum recovers the (coefficient, basis-vector) pairs in the first place—is the elementary generically-disjoint-cones argument flagged above, and is not re-treated here.

Reading on

The rest of this section makes good on Theorem 7.1. It rebuilds the construction in self-contained combinatorial language (Section 7.1), isolates the precise question (§7.2), and proves the local-equals-global equivalence from first principles, the whole turning on a single idea—*a snapshot remembers how it was built*—read once shallowly (to neutralise *clean*) and once fully (to neutralise $p \neq q$), with monotonicity then upgrading "each snapshot is alignable" to "one σ aligns them all" (§7.6). A reader who accepts the dictionary above may skim from §7.2 and rejoin at the worked examples that close the proof.

7.1 Definitions

Where n and k appear they may always be assumed to be integers greater than or equal to 2.

Let $s(n)$ be the set of length-$(n-1)$ ordered sequences of integers (without repeats) chosen from $[1, n]$. Equivalently, each element of $s(n)$ is obtained by leaving out exactly one of the n symbols and writing the remaining $n-1$ in some order (which symbol is left out will matter later). For example

$$s(3) = \{(1,2),(2,1),(1,3),(3,1),(2,3),(3,2)\}.$$

Clearly $|s(n)| = {}^nP_{n-1} = n!$.

Let $oh(\vec{v})$ be a function which takes an element of $\vec{v} \in s(n)$ to an ordered list of one-hot length-n vectors (a *one-hot* vector being one that is 1 in a single position and 0 everywhere else) defined using the Kronecker Delta as follows:

$$oh(\vec{v}) = (\vec{o}_1(\vec{v}), \vec{o}_2(\vec{v}), ..., \vec{o}_{n-1}(\vec{v}))$$

with

$$(\vec{o}_i(\vec{v}))_j = \delta_{j\vec{v}_i}.$$

For example: $oh((3,1)) = ((0,0,1),(1,0,0))$ and $oh((1,2)) = ((1,0,0),(0,1,0))$.

Let $coh(\vec{v})$ be a 'cumulative' version of $oh(\vec{v})$, that is to say:

$$(coh(\vec{v}))_i = \sum_{j=1}^{i} (oh(\vec{v}))_j.$$

For example: $coh((3,1)) = ((0,0,1),(1,0,1))$ and $coh((1,2)) = ((1,0,0),(1,1,0))$.

Let $mat(\vec{x}, j)$ be a function which inserts the elements of $\vec{x} \in \mathbb{Z}^n$ into the j-th row of a $k \times n$ matrix which otherwise contains zeros. For example, if $n = 4$ and $k = 3$ we would have

$$mat((2,4,3,9),2) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 2 & 3 & 4 & 9 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

For $\vec{v}_i \in s(n)$ and $i \in [1,k]$ let $U(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k)$ be the following set of $k \times n$-matrices:

$$U(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k) = \bigcup_{j=1}^{k} \bigcup_{m=1}^{n-1} \{mat(coh(\vec{v}_j)_m, j)\}.$$

For example, for $n = 3$ and $k = 2$ we would have

$$U((3,1),(1,2)) = \left\{ \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix} \right\}.$$

Note that $|U(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k)| = (n-1)k$ and so if we wished to put the elements of the set $U(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k)$ into some fixed order there would be $((n-1)k)!$ orderings available to us.

For p being an integer in $[1, ((n-1)k)!]$ let $\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p)$ be an ordering of the elements of $U(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k)$. (Think of p simply as a *name* for one particular ordering: there are $((n-1)k)!$ orderings, and p ranges over labels for them.) The only requirement on the ordering is that

$$\left(\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p) = \vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; q)\right) \iff (p = q),$$

i.e. that distinct values of p select different permuations. For example, it might be the case that

$$\vec{U}((3,1),(1,2);1) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix} \right)$$

while

$$\vec{U}((3,1),(1,2);2) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \right).$$

Let $C\vec{U}$ be a cumulative version of $\vec{U}$, i.e. for i in $[1, (n-1)k]$ we define

$$\left(C\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p)\right)_i = \sum_{j=1}^{i} \left(\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p)\right)_j.$$

For example, if

$$\vec{U}((3,1),(1,2);3) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \right)$$

then

$$C\vec{U}((3,1),(1,2);3) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix} \right).$$

Let a 'filter', $\vec{f}$, be a length-$(n-1)k$-vector consisting of only zeros or ones that can 'hide' elements of $C\vec{U}$ to make a filtered ordered list $FC\vec{U}$ as follows:

$$\left(FC\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p, \vec{f})\right)_i = \begin{cases} \left(C\vec{U}(\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_k; p)\right)_i & \text{if } (\vec{f})_i = 1, \\ Null & \text{otherwise.} \end{cases}$$

For example, if the filter is $\vec{f} = (1,1,0,1)$ and

$$C\vec{U}((3,1),(1,2);3) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix} \right) \tag{1}$$

then

$$FC\vec{U}((3,1),(1,2);3,(1,1,0,1)) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, Null, \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix} \right).$$

Finally, let $clean(\vec{L})$ be a function which takes an ordered list $\vec{L}$ (some of whose entries may be $Null$) and returns the shorter list obtained by deleting every $Null$ and closing up the gaps, leaving the surviving matrices in their original relative order. For example, continuing the case above,

$$clean\left(FC\vec{U}((3,1),(1,2);3,(1,1,0,1))\right) = \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 0 \end{bmatrix} \right).$$

Note that *clean* discards the knowledge of *where* the $Null$s sat: it preserves the surviving matrices' order relative to one another, but not the absolute positions $(1, 2, 3, \ldots)$ they originally occupied in the list.

A useful observation: $C\vec{U}$ and $FC\vec{U}$ are really sets

Each cumulative matrix quietly records its own position. The largest entry in a row counts how many of that row's pieces have been added in so far, so summing the largest entry over all rows returns the matrix's position in the list. For the third matrix of $C\vec{U}((3,1),(1,2);3)$ above,

$$m_3 = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \qquad \underbrace{\max(1,0,2)}_{2} + \underbrace{\max(1,1,0)}_{1} = 3,$$

which is indeed where it sits. Consequently the order in which the matrices of $C\vec{U}$ are listed – and, for $FC\vec{U}$, where the $Null$s fall – can always be reconstructed from the matrices themselves. **We may therefore treat $C\vec{U}$ and $FC\vec{U}$ as *sets* of matrices**, and will do so freely below; nothing we ask of them depends on their stored order. (The intermediate object $\vec{U}$ is different – its plain one-hot matrices do not betray their positions – so for $\vec{U}$ the listed order is genuine.)

7.1.1 Placing columns into canonical orders – (a.k.a. column 'sorting')

Before describing the two sorting processes we need one piece of notation. A *permutation* σ of $[1,n]$ is a relabelling of the column positions $\{1,2,\ldots,n\}$ – a one-to-one correspondence of that set with itself. (The set of all $n!$ such permutations is written S_n.) For a $k \times n$ matrix m, let $colPerm(m,\sigma)$ denote the matrix obtained by moving the contents of column c to column $\sigma(c)$, simultaneously in every row. For example, if σ sends $1 \mapsto 2,\ 2 \mapsto 3,\ 3 \mapsto 1$ then

$$colPerm\left(\begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \sigma\right) = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

(the old column 1 has gone to position 2, column 2 to position 3, and column 3 to position 1).

Suppose now that $\vec{L} = (m_1, m_2, \ldots, m_{(n-1)k})$ is an ordered list wherein each element m_u is either a $k \times n$-matrix or a $Null$ value (representing a filtered-away matrix). One can imagine two different (but related) column re-ordering processes which could be applied to the matrices in $\vec{L}$. Call these:

1. Local column sorting, and
2. Global column sorting.

Local column sorting. Here each matrix is tidied *on its own* (*local canonicalisation*, the embedder's Step E). We suppose we have some fixed rule $canon(\cdot)$ that takes a matrix to a chosen "standard" column-ordering of it, and we apply it to each matrix independently:

$$localSort(\vec{L}) = \big(canon(m_1), canon(m_2), \ldots, canon(m_{(n-1)k})\big)\,.$$

We deliberately do *not* pin down which standard ordering *canon* produces, because nothing in this document depends on the choice: a companion document that uses this machinery is free to pick its own rule, and may even change it from time to time. The *only* property we ever require of *canon* is that it be a genuine canonical form – that it give two matrices the same output exactly when they are column-reorderings of one another:

$$canon(a) = canon(b) \quad \Longleftrightarrow \quad a = colPerm(b,\sigma) \text{ for some } \sigma. \tag{2}$$

(For concreteness, one rule obeying (2) is: read each matrix's entries off in a fixed order, say row by row, and among all $n!$ column-reorderings keep the one whose readout is alphabetically smallest. But this is only an *example*; any rule satisfying (2) serves equally well, and we rely on no other feature of it.)

Global column sorting. Here, in contrast, a *single* column permutation σ must be applied synchronously to *all* the matrices at once, and only then is the whole list tidied to a standard form. We suppose we have some fixed rule $globalSort(\cdot)$ that selects a standard representative from among all the lists

$$\big(colPerm(m_1,\sigma), colPerm(m_2,\sigma), \ldots, colPerm(m_{(n-1)k},\sigma)\big), \qquad \sigma \in S_n,$$

reachable by re-shuffling the columns of every matrix by one common σ. As with *canon* we do not commit to *how* the representative is chosen; the only property we require is the exact analogue of (2), one level up – that two lists receive the same *globalSort* precisely when one can be turned into the other by a single common column permutation:

$$globalSort(\vec{L}) = globalSort(\vec{L}') \iff \begin{array}{l} m'_i = colPerm(m_i,\sigma) \\ \text{for all } i, \text{ for one common } \sigma. \end{array} \tag{3}$$

The contrast between the two is the whole point. Local sorting is allowed a *different* reshuffle for each matrix; global sorting must make *one* reshuffle serve them all. Global agreement is therefore the harder thing to achieve, and the question of Section 7.2 is whether it is nonetheless forced by local agreement.

7.2 The problem we really want to solve

Everything in Section 7.1 was scaffolding. Here is the question we actually care about.

Throughout, fix two *configurations*

$$\vec{v} = (\vec{v}_1, \ldots, \vec{v}_k) \qquad \text{and} \qquad \vec{u} = (\vec{u}_1, \ldots, \vec{u}_k), \qquad \vec{v}_j, \vec{u}_j \in s(n).$$

Each is just a list of k sequences of the kind defined in Section 7.1. Think of $\vec{v}$ as one "experiment" and $\vec{u}$ as another, and the question we ask is always: *can we tell these two experiments apart* after we have processed them in the elaborate way Section 7.1 describes?

The problem. We wish to know whether, for *all* filters $\vec{f}$ and *all* orderings p and q,

$$\begin{gathered} \Big(globalSort\big(clean(FC\vec{U}(\vec{v};p,\vec{f}))\big) = globalSort\big(clean(FC\vec{U}(\vec{u};q,\vec{f}))\big)\Big) \\ \iff \\ \Big(localSort\big(clean(FC\vec{U}(\vec{v};p,\vec{f}))\big) = localSort\big(clean(FC\vec{U}(\vec{u};q,\vec{f}))\big)\Big). \end{gathered} \tag{4}$$

Either we need a proof that (4) always holds, or a counterexample.

What the two sides mean, in words. Both sides take the same raw ingredients, filter them with the same $\vec{f}$, throw away the gaps, and then try to line up the columns of the resulting matrices into a canonical order. They differ only in *how much freedom* the column re-ordering is given (this is the distinction between *local* and *global* column sorting drawn at the end of Section 7.1):

- **localSort** canonicalises *each matrix on its own*: every matrix may choose its own column permutation.
- **globalSort** must canonicalise *all the matrices at once* using a *single* column permutation applied to every matrix simultaneously.

Global agreement is therefore the stronger, more demanding statement. The content of (4) is the claim that, surprisingly, the weaker "each-matrix-separately" notion of agreement is already enough to force the stronger "all-at-once" one. We will prove that it is.

Theorem 7.1. *For every pair of configurations $\vec{v}, \vec{u}$, every filter $\vec{f}$, and all orderings p, q, the biconditional (4) holds.*

First simplification: the lists are secretly sets

The statement (4) looks forbidding because of the four wrappers (*globalSort*/*localSort*, *clean*, $FC\vec{U}$, and the orderings p, q). The first thing to do is to dissolve most of them.

The key was already noticed in Section 7.1: *a cumulative matrix records its own position in the list.* If m_i is the i-th cumulative matrix, then summing the largest entry in each of its k rows returns i:

$$i = \sum_{r=1}^{k} \max_{c \in [1,n]} (m_i)_{rc}. \tag{5}$$

For example, with $C\vec{U}((3,1),(1,2);3)$ from Section 7.1 the third matrix is

$$m_3 = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}, \qquad \underbrace{\max(1,0,2)}_{2} + \underbrace{\max(1,1,0)}_{1} = 3,$$

which is exactly its position.

Two consequences follow, and together they remove three of the four wrappers.

- **Cleaning loses nothing.** Because each surviving matrix announces its own absolute position through (5), the positions of the discarded $Null$s can be reconstructed from the survivors alone. Whatever information *clean* appears to destroy was never really lost.
- **Order is redundant.** For the same reason, the order in which the matrices are listed can always be recovered from the matrices themselves. So we may treat each of $C\vec{U}$, $FC\vec{U}$ and its cleaned form not as an *ordered list* but simply as a *set* of matrices.

Corollary 7.2. *$C\vec{U}(\vec{v};p)$ and $FC\vec{U}(\vec{v};p,\vec{f})$ may be regarded as* sets *of matrices: the order intended for their elements is always recoverable from the elements themselves via* (5)*. Consequently clean has no effect on the underlying set, and the equalities in* (4) *are equalities of* sets *of matrices.*

There is one subtlety to check before we lean on this. Both *localSort* and *globalSort* act by *permuting columns*, and permuting the columns of a row never changes *which numbers* appear in that row – only their left-to-right arrangement. In particular the largest entry of each row is unchanged, so the position formula (5) keeps working *after* sorting as well as before. Hence a sorted, cleaned list is still recoverable from its set of matrices, and "the two sorted lists are equal" means exactly "the two sets of sorted matrices are equal". This is what lets us speak of sets from now on.

What remains. Stripped of *clean* and of the ordering bookkeeping, the problem is this. Each side produces a *set of matrices*. We must compare

- the set obtained by sorting *each matrix's columns independently* (*localSort*), against
- the set obtained by sorting *all matrices' columns with one common permutation* (*globalSort*),

and show the two notions of "the $\vec{v}$-set equals the $\vec{u}$-set" coincide. The rest of the document does this. The plan is:

§7.3 introduces a cleaner name – a *snapshot* – for the matrices that appear, indexed not by a position but by the set of ingredients that built it;

§7.4 proves that a snapshot *remembers exactly which ingredients built it* (formula (5) is the shallow end of this);

§7.5 settles, for a single snapshot, exactly when the $\vec{v}$-version and $\vec{u}$-version are column reorderings of each other;

§7.6 assembles these into a proof of Theorem 7.1.

7.3 Snapshots

This section renames the objects of Section 7.1 in a way that makes the proof short. No new mathematics happens here; we are only choosing better bookkeeping.

Slots and slot-sets

The raw ingredients of Section 7.1 – the elements of the set $U(\vec{v}_1, \ldots, \vec{v}_k)$ – are indexed by two numbers: a *row* $j \in [1, k]$ and a *level* $m \in [1, n-1]$. Call the pair (j, m) a *slot*. There are exactly $L = (n-1)k$ slots, one for each ingredient. The slot (j, m) names the ingredient "put the cumulative one-hot $coh(\vec{v}_j)_m$ into row j, zeros elsewhere".

A *slot-set* Q is just any set of slots, $Q \subseteq \{(j, m)\}$. Given a slot-set Q, gather for each row j the levels it contains:

$$A_j(Q) = \{\, m : (j, m) \in Q \,\} \subseteq [1, n-1].$$

So Q is completely described by its k level-sets $A_1(Q), \ldots, A_k(Q)$.

Prefix sets

For a sequence $\vec{w} \in s(n)$ and a level m, write

$$P_m(\vec{w}) = \{\vec{w}_1, \vec{w}_2, \ldots, \vec{w}_m\}$$

for the *prefix set*: the set of the first m symbols of $\vec{w}$ (forgetting their order). Because $\vec{w}$ has no repeats, these grow one symbol at a time and are *nested*,

$$P_1(\vec{w}) \subset P_2(\vec{w}) \subset \cdots \subset P_{n-1}(\vec{w}).$$

The cumulative one-hot vector $coh(\vec{w})_m$ from Section 7.1 is precisely the *indicator* of $P_m(\vec{w})$ – the length-n row of 0s and 1s with a 1 in column c exactly when $c \in P_m(\vec{w})$. For instance with $\vec{w} = (3, 1)$ we have $P_1 = \{3\}$, $P_2 = \{3, 1\}$, and the indicators $(0, 0, 1)$ and $(1, 0, 1)$.

The snapshot matrix

Definition 7.3. *For a configuration $\vec{v} = (\vec{v}_1, \ldots, \vec{v}_k)$ and a slot-set Q, the* snapshot $S^{\vec{v}}(Q)$ *is the $k \times n$ matrix whose row j is the sum of the indicators of the prefixes named by Q:*

$$\big(S^{\vec{v}}(Q)\big)_{row\ j} = \sum_{m \in A_j(Q)} coh(\vec{v}_j)_m.$$

This is nothing more than "add up the ingredients whose slots lie in Q". The link to Section 7.1 is immediate: if Q_i denotes the set of the *first i slots* in an ordering p, then $S^{\vec{v}}(Q_i)$ is exactly the i-th cumulative matrix of $C\vec{U}(\vec{v}; p)$. The word "snapshot" is apt: $S^{\vec{v}}(Q_i)$ is a frozen moment of the running total after i steps. But the definition makes sense for *any* slot-set Q, prefix or not, and that extra generality costs nothing.

A worked snapshot. Take $n = 3$, $k = 2$, $\vec{v} = ((3, 1), (1, 2))$ – the running example of Section 7.1 – and the slot-set $Q = \{(1, 1), (1, 2), (2, 2)\}$, so that $A_1(Q) = \{1, 2\}$ and $A_2(Q) = \{2\}$. Row 1 is $coh(\vec{v}_1)_1 + coh(\vec{v}_1)_2 = (0, 0, 1) + (1, 0, 1) = (1, 0, 2)$, and row 2 is $coh(\vec{v}_2)_2 = (1, 1, 0)$. Thus

$$S^{\vec{v}}(Q) = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}.$$

The values depend on Q alone; the configuration only moves them around

We first fix one small piece of notation. For a sequence $\vec{w} \in s(n)$ write $(\vec{w})_r$ for its r-th entry – the symbol in position r – so that "column $(\vec{w})_r$" means the column belonging to that symbol. As r runs $1, \dots, n-1$ these name the $n-1$ columns that $\vec{w}$ uses; the single left-out symbol owns the one remaining column.

Here is the observation that makes the whole problem tractable. Look at row j of a snapshot. The prefixes are nested, so the entry in column $(\vec{v}_j)_r$ simply counts how many of the chosen levels reach that far:

$$\big(S^{\vec{v}}(Q)\big)_{j,\,(\vec{v}_j)_r} = \#\{\, m \in A_j(Q) : m \geq r \,\},$$

and the one column belonging to the omitted symbol gets a 0 (recall that each sequence $\vec{v}_j$ lists only $n-1$ of the n symbols, so exactly one symbol – and hence one column – never appears). The crucial point: the *multiset of values* appearing in row j – namely $\{\#\{m \in A_j(Q) : m \geq r\} : r\}$ together with the extra 0 – depends only on $A_j(Q)$, **not** on the sequence $\vec{v}_j$. (A *multiset* is a collection in which repeats are allowed and counted but order is ignored: thus $\{2,1,0\}$ and $\{0,1,2\}$ are the same multiset, whereas $\{1,1,0\}$ differs from $\{1,0\}$.) The sequence decides only *which column* each value lands in.

Same values, different columns. With $A_1(Q) = \{1,2\}$ the row-1 values are $\#\{m \geq 1\} = 2$, $\#\{m \geq 2\} = 1$, plus a 0: the multiset $\{2,1,0\}$, whatever the sequence. If $\vec{v}_1 = (3,1)$ then 2 goes to column 3, 1 to column 1, 0 to column 2, giving row $(1,0,2)$. If instead the sequence were $(2,3)$ the *same* values $\{2,1,0\}$ would sit in different columns, giving row $(0,2,1)$. Same contents; different arrangement.

This single fact – *contents fixed by Q, arrangement fixed by the sequence* – is the seed of both halves of the proof. Reading off the contents will tell us Q (Section 7.4); comparing the arrangements will tell us when two configurations agree (Section 7.5).

7.4 A snapshot remembers how it was built

We now prove that a snapshot matrix secretly carries a complete record of the slot-set that produced it. Formula (5) read off one number from each row; here we read off the whole row.

Lemma 7.4. *Fix a row j and let $A = A_j(Q)$. From the multiset of entries in row j of $S^{\vec{v}}(Q)$ one can recover the set A completely, and the recovery does not use the sequence $\vec{v}_j$. Explicitly, for every integer $t \geq 1$,*

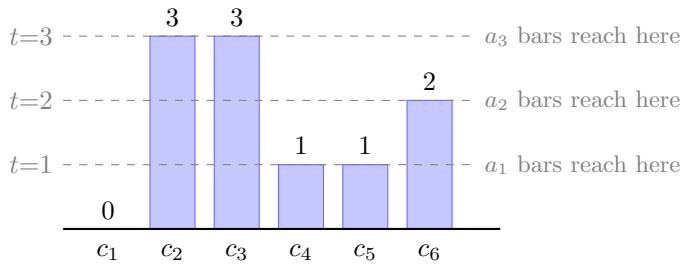
$$\#\{\, c : \big(S^{\vec{v}}(Q)\big)_{j,c} \geq t \,\} = \big(\textit{the } t\textit{-th largest element of } A\big),$$

and this count is 0 once $t > |A|$. Hence, listing $A = \{a_1 > a_2 > \cdots > a_{|A|}\}$, the numbers $a_1, a_2, \dots$ are read straight off the row.

Proof. By the displayed formula of Section 7.3, the entry in column $(\vec{v}_j)_r$ is $\#\{m \in A : m \geq r\}$ (and the absent column is 0). Such an entry is $\geq t$ precisely when its position r satisfies $r \leq a_t$, where a_t is the t-th largest element of A; there are exactly a_t such positions. So the number of columns with entry $\geq t$ equals a_t. These counts mention only A, never the sequence, and running $t = 1, 2, \dots$ recovers $a_1, a_2, \dots$, i.e. all of A. □

Example. Row 1 of the snapshot above was $(1,0,2)$. Columns with entry ≥ 1: two of them, so $a_1 = 2$; with entry ≥ 2: one, so $a_2 = 1$; with entry ≥ 3: none, so we stop. Thus $A_1(Q) = \{2,1\} = \{1,2\}$, exactly the levels we put in – recovered from the row alone, with no reference to the sequence $(3,1)$.

The reading is easiest to see as a picture, and is clearer in a less trivial case. Suppose some snapshot row – in an example with $n = 6$, so columns $c_1, \dots, c_6$ – reads $(0,3,3,1,1,2)$. Draw its entries as bar heights and rule in the thresholds $t = 1, 2, 3$:



Counting the bars that reach each dashed line gives $a_1 = 5$ (entries ≥ 1), $a_2 = 3$ (entries ≥ 2), $a_3 = 2$ (entries ≥ 3), and nothing reaches higher; so the row must have been built from the level-set $A = \{5, 3, 2\}$. Notice we never needed to know *which* sequence produced the row: the bar heights alone give back A.

Two corollaries do all the work later.

Corollary 7.5 (the whole slot-set is remembered)**.** *The entire slot-set Q can be recovered from the matrix $S^{\vec{v}}(Q)$: apply Lemma 7.4 to each row j to get $A_j(Q)$, then $Q = \{(j, m) : m \in A_j(Q)\}$. In particular two snapshots built from* different *slot-sets are different matrices, whatever the configurations.*

Corollary 7.6 (remembering survives column sorting)**.** *Permuting the columns of a snapshot does not change any row's multiset of entries, so Q is still recoverable afterwards. In particular Q can be read from localSort's or globalSort's output just as from the original.*

Formula (5) is just the special case $t = 1$: the count of nonzero entries in row j is $a_1 = |A_j(Q)|$, and summing over rows gives $|Q|$, the position. Lemma 7.4 simply keeps reading down the row instead of stopping at the top.

These corollaries are exactly what we need to defeat the two "extra freedoms" in the problem. Corollary 7.6 (with $t = 1$) was the content of Corollary 7.2: it lets us ignore *clean* and ordering and work with sets. Corollary 7.5 will let us ignore the fact that the two sides may use *different* orderings $p \neq q$: if a $\vec{v}$-snapshot and a $\vec{u}$-snapshot ever match (even after independent column sorting) they must have been built from one and the same slot-set.

7.5 When are two snapshots the same up to column order?

Fix a single slot-set Q and compare the two snapshots $S^{\vec{v}}(Q)$ and $S^{\vec{u}}(Q)$ built from it. By the previous section they have, row by row, the same multiset of values; they can differ only in *which columns* hold those values. When is one merely a column-rearrangement of the other?

Vocabulary: permutations and column equivalence

Recall from Section 7.1 that a *permutation* $\sigma \in S_n$ relabels the columns $[1, n]$, and that $colPerm(M, \sigma)$ is the matrix got by sending the contents of column c to column $\sigma(c)$ in every row. Two matrices are *column-equivalent*, written $M \sim M'$, if some single σ achieves $colPerm(M, \sigma) = M'$ – that is, if one is a column-rearrangement of the other. This is exactly the relation that *canon* detects: by (2), $canon(M) = canon(M')$ iff $M \sim M'$.

Aligning the revealed prefixes

Recall (Section 7.3) that reading row j of $S^{\vec{v}}(Q)$ at "height $\geq t$" returns a prefix set $P_{a_t}(\vec{v}_j)$. As t ranges over $1, \ldots, |A_j(Q)|$ these are precisely the prefixes $\{P_m(\vec{v}_j) : m \in A_j(Q)\}$. So a snapshot is best thought of as a device that *displays*, in each row, a nested stack of prefix sets – one prefix for each level in $A_j(Q)$. Lining up two snapshots column-by-column therefore means lining up these displayed prefixes. Define

$$\Sigma(Q) = \big\{\, \sigma \in S_n : \ \sigma\big(P_m(\vec{v}_j)\big) = P_m(\vec{u}_j) \text{ for all } j \in [1, k],\ m \in A_j(Q) \,\big\},$$

the relabellings that carry every prefix revealed by $\vec{v}$ onto the matching prefix revealed by $\vec{u}$. (Here $\sigma(S)$ means $\{\sigma(c) : c \in S\}$.)

Lemma 7.7 (alignment)**.** *$S^{\vec{v}}(Q) \sim S^{\vec{u}}(Q)$ if and only if $\Sigma(Q) \neq \varnothing$. More precisely, $\Sigma(Q)$ is* exactly *the set of column permutations σ with $colPerm(S^{\vec{v}}(Q), \sigma) = S^{\vec{u}}(Q)$.*

Proof. A matrix of non-negative integers is determined by its family of "level sets" $\{c : \text{entry} \geq t\}$ in each row (the set of columns whose entry clears height t, for each threshold $t = 1, 2, \ldots$). In row j of $S^{\vec{v}}(Q)$ that level set is the prefix $P_{a_t}(\vec{v}_j)$, with a_t the t-th largest element of $A_j(Q)$; and crucially $A_j(Q)$ is the *same* on the $\vec{u}$-side, so the thresholds a_t match. Applying $colPerm(\cdot, \sigma)$ sends each level set S to $\sigma(S)$. Hence $colPerm(S^{\vec{v}}(Q), \sigma)$ equals $S^{\vec{u}}(Q)$ exactly when, in every row j and at every threshold, $\sigma\big(P_{a_t}(\vec{v}_j)\big) = P_{a_t}(\vec{u}_j)$ – that is, when $\sigma(P_m(\vec{v}_j)) = P_m(\vec{u}_j)$ for all j and all $m \in A_j(Q)$, which is the definition of $\sigma \in \Sigma(Q)$. □

Example (a relabelling, and a non-relabelling). Keep $n = 3$, $k = 2$, $\vec{v} = ((3,1),(1,2))$ and $Q = \{(1,1),(1,2),(2,2)\}$, so

$$S^{\vec{v}}(Q) = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 1 & 0 \end{bmatrix}.$$

Take $\vec{u} = ((2,3),(3,1))$, obtained from $\vec{v}$ by the single relabelling $\sigma : 1 \mapsto 3,\ 2 \mapsto 1,\ 3 \mapsto 2$ applied to every symbol. Then

$$S^{\vec{u}}(Q) = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix} = colPerm\Big(S^{\vec{v}}(Q), \sigma\Big),$$

so $S^{\vec{v}}(Q) \sim S^{\vec{u}}(Q)$ and indeed $\sigma \in \Sigma(Q)$: one relabelling reorders the columns of *both* rows at once. By contrast take $\vec{w} = ((1,3),(2,1))$, which is *not* a relabelling of $\vec{v}$. Lining up the displayed prefixes would force, from row 1 at level 1, that $\sigma(\{3\}) = \{1\}$, i.e. $\sigma(3) = 1$; yet row 2 at level 2 forces $\sigma(\{1,2\}) = \{2,1\}$, i.e. σ maps $\{1,2\}$ onto $\{1,2\}$ – so the symbol 1 would have to be the image of 3 *and* of one of $1, 2$ at once, which no bijection allows. Hence $\Sigma(Q) = \varnothing$ and the two snapshots are genuinely different no matter how their columns are shuffled.

The richer the snapshot, the fewer alignments survive

A snapshot built from more slots displays more prefixes, hence imposes more constraints on an aligning σ. This monotonicity is the engine of the whole argument.

Corollary 7.8 (monotonicity)**.** *If $Q' \subseteq Q$ then $\Sigma(Q) \subseteq \Sigma(Q')$.*

Proof. A smaller slot-set has smaller level-sets, $A_j(Q') \subseteq A_j(Q)$ for every j. The conditions defining $\Sigma(Q')$ are therefore a sub-list of those defining $\Sigma(Q)$. Adding requirements can only shrink the solution set, so any σ in $\Sigma(Q)$ already lies in $\Sigma(Q')$. □

7.6 Putting it together

We can now prove Theorem 7.1. Fix $\vec{v}, \vec{u}, \vec{f}, p, q$.

The retained slot-sets

Let Q_i^p be the set of the first i slots in ordering p. These are *nested*, $Q_1^p \subset Q_2^p \subset \cdots$, so they form what is called a *chain*: any two of them are comparable, one contained in the other. The filter $\vec{f}$ keeps some of them; write

$$\mathcal{R}_{\vec{v}} = \{\, Q_i^p : (\vec{f})_i = 1 \,\}, \qquad \mathcal{R}_{\vec{u}} = \{\, Q_i^q : (\vec{f})_i = 1 \,\}$$

for the families of *retained* slot-sets on the two sides. Each is a chain (a sub-collection of a chain), so if non-empty it has a largest member.

By Corollary 7.2 the cleaned, sorted output of each side is just a *set* of matrices, namely

$$\{\, \mathit{canon}\, S^{\vec{v}}(Q) : Q \in \mathcal{R}_{\vec{v}} \,\} \quad \text{(local, } \vec{v} \text{ side)},$$

and similarly for $\vec{u}$; for the global versions one common σ is applied first. We must compare these.

Local equality, decoded

Lemma 7.9. *localSort of the two sides agree if and only if*

$$\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}} \ (=: \mathcal{R}) \qquad \textit{and} \qquad \Sigma(Q) \neq \varnothing \textit{ for every } Q \in \mathcal{R}.$$

Proof. *localSort* agreement means the two sets $\{\mathit{canon}\, S^{\vec{v}}(Q) : Q \in \mathcal{R}_{\vec{v}}\}$ and $\{\mathit{canon}\, S^{\vec{u}}(Q') : Q' \in \mathcal{R}_{\vec{u}}\}$ are equal. By Corollary 7.6, each matrix $\mathit{canon}\, S^{\vec{v}}(Q)$ still reveals the slot-set Q that built it; likewise on the $\vec{u}$-side. So a matrix common to both sets carries one slot-set, read the same way from either description: the Q from the $\vec{v}$-side and the Q' from the $\vec{u}$-side that produced it must be *equal*. Matching the two sets element by element therefore matches each $Q \in \mathcal{R}_{\vec{v}}$ with the *same* $Q \in \mathcal{R}_{\vec{u}}$, forcing $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}}$, and for each such Q the common matrix says $\mathit{canon}\, S^{\vec{v}}(Q) = \mathit{canon}\, S^{\vec{u}}(Q)$, i.e. $S^{\vec{v}}(Q) \sim S^{\vec{u}}(Q)$. By Lemma 7.7 this is $\Sigma(Q) \neq \varnothing$. Every step is reversible, giving the converse. □

This already contains the surprise that the two orderings p and q are forced into step: local agreement cannot hold unless $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}}$, i.e. at every retained position the two sides have gathered *identical* slot-sets, even though they were free to use different orderings.

Global equality, decoded

Lemma 7.10. *globalSort of the two sides agree if and only if*

$$\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}} \ (=: \mathcal{R}) \qquad \textit{and} \qquad \bigcap_{Q \in \mathcal{R}} \Sigma(Q) \neq \varnothing.$$

Proof. ($\Leftarrow$) Suppose the families agree and some single σ lies in every $\Sigma(Q)$, $Q \in \mathcal{R}$. By Lemma 7.7, σ turns each $S^{\vec{v}}(Q)$ into $S^{\vec{u}}(Q)$. So the whole list of $\vec{u}$-snapshots is obtained from the whole list of $\vec{v}$-snapshots by that one common column permutation σ. Two lists related by a single common column permutation are tidied to the *same* thing by *globalSort* – that is exactly the freedom *globalSort* removes, property (3) – so the two *globalSort* outputs agree.

($\Rightarrow$) Suppose the *globalSort* outputs are equal. By property (3), the two (cleaned) lists must then be related by a *single* common column permutation σ: reading them off in their shared order, each $\vec{u}$-snapshot equals *colPerm* of the corresponding $\vec{v}$-snapshot by this one σ. A column permutation preserves every row's multiset, so by Corollary 7.5 the paired snapshots share a slot-set; matching them up gives $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}}$, and the relation $\mathit{colPerm}(S^{\vec{v}}(Q), \sigma) = S^{\vec{u}}(Q)$ for every retained Q says exactly (Lemma 7.7) that this one σ lies in $\bigcap_Q \Sigma(Q)$. □

The two conditions coincide

Compare Lemmas 7.9 and 7.10. Both demand $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}} = \mathcal{R}$; granting that, local agreement asks $\Sigma(Q) \neq \varnothing$ for *each* retained Q, while global agreement asks the single intersection $\bigcap_{Q \in \mathcal{R}} \Sigma(Q)$ to be non-empty. We finish by showing these last two are the same condition.

If $\mathcal{R}$ is empty, both sorted lists are empty and there is nothing to prove. Otherwise $\mathcal{R}$ is a non-empty chain, so it has a largest member

$$Q^* = \text{the richest retained slot-set.}$$

Because every $Q \in \mathcal{R}$ satisfies $Q \subseteq Q^*$, monotonicity (Corollary 7.8) gives $\Sigma(Q^*) \subseteq \Sigma(Q)$ for every such Q. Hence the intersection is controlled entirely by its richest member:

$$\bigcap_{Q \in \mathcal{R}} \Sigma(Q) = \Sigma(Q^*).$$

Now both sides of the biconditional are visible at once:

$$\underbrace{\big(\forall Q \in \mathcal{R} :\ \Sigma(Q) \neq \varnothing\big)}_{\text{local agreement}} \iff \Sigma(Q^*) \neq \varnothing \iff \underbrace{\Big(\bigcap_{Q \in \mathcal{R}} \Sigma(Q) \neq \varnothing\Big)}_{\text{global agreement}}.$$

The left equivalence holds because Q^* is itself one of the Q's (so the "for all" implies the middle), and conversely $\Sigma(Q^*) \subseteq \Sigma(Q)$ makes a non-empty $\Sigma(Q^*)$ force every $\Sigma(Q)$ non-empty; the right equivalence is the displayed equality. Since both notions of agreement also required the same clause $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}}$, they coincide. This proves Theorem 7.1. □

The proof in one breath

Among the retained snapshots there is a richest one, Q^*. It displays every prefix that any other retained snapshot displays, so the relabellings that align it are already all the relabellings that align everything (monotonicity). Local agreement says merely that this richest snapshot can be aligned *on its own*; but "aligning the richest one" is the same as "aligning all of them at once", which is global agreement. The two extra freedoms in the problem evaporate for one reason each: a snapshot remembers its own slot-set, so dropping the $Null$s (Corollary 7.2) and using different orderings (Corollary 7.5) change nothing.

A worked example

Let us watch the whole machine run once, on an example rich enough to be interesting. Take $n = 4$ and $k = 2$, with

$$\vec{v}_1 = (3,1,4), \quad \vec{v}_2 = (4,2,1), \qquad \vec{u}_1 = (1,2,3), \quad \vec{u}_2 = (3,4,2).$$

Here $\vec{u}$ is $\vec{v}$ relabelled by the single permutation $\sigma : 1 \mapsto 2,\ 2 \mapsto 4,\ 3 \mapsto 1,\ 4 \mapsto 3$ (apply σ to every symbol of each $\vec{v}_j$ to obtain $\vec{u}_j$).

Two different orderings, one filter. There are $(n-1)k = 6$ slots. Let the two sides use *different* orderings

$$\begin{aligned} p :&\ (1,1),(2,1),(1,2),(2,2),(1,3),(2,3),\\ q :&\ (2,1),(1,1),(2,2),(1,2),(2,3),(1,3) \end{aligned}$$

(so $p \neq q$: they swap the two slots within each consecutive pair), and the common filter $\vec{f} = (0,1,0,1,0,1)$, which keeps only positions $2,4,6$. The retained slot-sets are the prefixes of length $2,4,6$; and although p and q disagree on the *odd*-length prefixes, they *agree* on these even ones:

$$R_1 = \{(1,1),(2,1)\}, \qquad R_2 = R_1 \cup \{(1,2),(2,2)\}, \qquad R_3 = \text{all six slots}.$$

Hence $\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}} = \{R_1 \subset R_2 \subset R_3\}$, a chain whose richest member is $Q^* = R_3$. (That the two sides share the same retained family is the synchronisation that local agreement forces in general, by Lemma 7.9; we will see at the end what goes wrong if it fails.)

The retained snapshots. Filling each row j with $\sum_{m \in A_j(R)} coh(\cdot)_m$ (the omitted symbol's column staying 0) gives:

	$\vec{v}$ side	$\vec{u}$ side
R_1	$S^{\vec{v}}(R_1) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$S^{\vec{u}}(R_1) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$
R_2	$S^{\vec{v}}(R_2) = \begin{bmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \end{bmatrix}$	$S^{\vec{u}}(R_2) = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 0 & 0 & 2 & 1 \end{bmatrix}$
R_3	$S^{\vec{v}}(R_3) = \begin{bmatrix} 2 & 0 & 3 & 1 \\ 1 & 2 & 0 & 3 \end{bmatrix}$	$S^{\vec{u}}(R_3) = \begin{bmatrix} 3 & 2 & 1 & 0 \\ 0 & 1 & 3 & 2 \end{bmatrix}$

(The rows now carry genuinely varied values: $0, 1, 2, 3$ all appear.)

Local agreement, and the alignment sets Σ. Reading off the revealed prefixes, the relabellings that line up the $\vec{v}$- and $\vec{u}$-snapshots are

$$\Sigma(R_1) = \{\sigma : \ \sigma(3) = 1, \ \sigma(4) = 3\},$$
$$\Sigma(R_2) = \Sigma(R_3) = \{\sigma : \ \sigma(3) = 1, \ \sigma(1) = 2, \ \sigma(4) = 3, \ \sigma(2) = 4\}.$$

The first allows *two* permutations (the pair $\{\sigma(1), \sigma(2)\} = \{2, 4\}$ is still free); the second pins all four images, so it contains just the *one* permutation σ. Each is non-empty, so each $\vec{v}$-snapshot is a column-rearrangement of its $\vec{u}$-partner: **local agreement holds**. Notice the alignment sets shrink as the snapshot grows richer, $\Sigma(R_1) \supseteq \Sigma(R_2) \supseteq \Sigma(R_3)$ (here $2 \geq 1 \geq 1$) – this is monotonicity (Corollary 7.8) before our eyes.

Global agreement. The intersection $\bigcap_i \Sigma(R_i)$ is pinned by the richest set: it equals $\Sigma(R_3) = \{\sigma\}$, which is non-empty. So the *single* permutation $\sigma : 1 \mapsto 2, 2 \mapsto 4, 3 \mapsto 1, 4 \mapsto 3$ re-orders the columns of all three $\vec{v}$-snapshots into all three $\vec{u}$-snapshots simultaneously: **global agreement holds too**, exactly as the theorem predicts. As a check, $colPerm(S^{\vec{v}}(R_3), \sigma)$ sends the entry in old column 3 to new column 1, old 1 to new 2, old 4 to new 3 and old 2 to new 4, turning $\left[\begin{smallmatrix} 2 & 0 & 3 & 1 \\ 1 & 2 & 0 & 3 \end{smallmatrix}\right]$ into $\left[\begin{smallmatrix} 3 & 2 & 1 & 0 \\ 0 & 1 & 3 & 2 \end{smallmatrix}\right] = S^{\vec{u}}(R_3)$.

What would break it. Suppose the filter had instead kept position 1. There p and q disagree: the $\vec{v}$-side snapshot is built from $\{(1, 1)\}$ but the $\vec{u}$-side from $\{(2, 1)\}$,

$$S^{\vec{v}}(\{(1,1)\}) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \qquad S^{\vec{u}}(\{(2,1)\}) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

Their first rows already have different multisets ($\{1, 0, 0, 0\}$ versus $\{0, 0, 0, 0\}$), so no column shuffle can match them and local agreement fails at that very position. This is the recovery lemma biting: a snapshot remembers the slot-set that built it, so two snapshots from *different* slot-sets can never be made to look alike. Local agreement can therefore only hold where p and q have gathered the same pieces – and there, as we saw, it drags global agreement along with it.

A second worked example: tied columns, and why a broken tie never returns

The example above never produced two equal *non-zero* columns, which rightly prompts the question: can a snapshot have genuinely tied non-zero columns – and if so, could a tie that is *broken* high up the chain *re-form* lower down? That would make Σ *grow* as Q grows and wreck the nesting the proof depends on. Here is an example with real ties; it shows them *breaking* as we descend, and makes visible why they can never come back.

Take $n = 3$, $k = 2$, with

$$\vec{v}_1 = (1,2), \quad \vec{v}_2 = (2,1), \qquad \vec{u}_1 = (2,3), \quad \vec{u}_2 = (3,2),$$

where $\vec{u}$ is $\vec{v}$ relabelled by $\sigma : 1 \mapsto 2,\ 2 \mapsto 3,\ 3 \mapsto 1$. Use the single ordering $p:\ (1,2),(2,2),(1,1),(2,1)$ on both sides and keep everything ($\vec{f} = (1,1,1,1)$), so the retained chain is $Q_1 \subset Q_2 \subset Q_3 \subset Q_4$ with

$$Q_1 = \{(1,2)\}, \quad Q_2 = Q_1 \cup \{(2,2)\}, \quad Q_3 = Q_2 \cup \{(1,1)\}, \quad Q_4 = \text{all four.}$$

(Note p deliberately adds level 2 of each row *before* level 1 – legal, since an ordering may list the four slots however it likes.) The snapshots are

	$\vec{v}$ side	$\vec{u}$ side
Q_1	$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$
Q_2	$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$
Q_3	$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}$
Q_4	$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}$

Look at the $\vec{v}$ side. At Q_1 columns 1 and 2 are both $(1,0)$; at Q_2 they are both $(1,1)$ – the two tied $(1,1)$ columns in question. From Q_3 onwards the tie is gone: column 1 has become $(2,1)$ while column 2 stays $(1,1)$.

These ties are precisely the freedom in σ. Reading off the prefixes,

$$\Sigma(Q_1) = \Sigma(Q_2) = \{\sigma :\ \sigma(\{1,2\}) = \{2,3\}\} \quad (\textit{two} \text{ permutations}),$$
$$\Sigma(Q_3) = \Sigma(Q_4) = \{\sigma\} \quad (\textit{one} \text{ permutation}).$$

The two permutations available at Q_1, Q_2 are the genuine relabelling σ and the one that *also* swaps the two tied columns; the instant the tie breaks (at Q_3) that swap dies, leaving only σ. So down the chain $|\Sigma|$ runs $2,2,1,1$ – it never rises.

Why the tie cannot re-form. Watch columns 1 and 2, i.e. symbols 1 and 2. In row 1 (sequence $\vec{v}_1 = (1, 2)$) symbol 1 sits at position 1 and symbol 2 at position 2, so their row-1 value-gap is

$$\#\{\, m \in A_1(Q) :\ 1 \le m < 2 \,\} = \#\{\, m \in A_1(Q) :\ m = 1 \,\},$$

which is 0 until level 1 enters row 1 (it does so at Q_3) and 1 thereafter. Since $A_1(Q)$ only *grows* as we descend, once that count reaches 1 it stays ≥ 1: the gap, once opened, never closes. The same holds in every row. Two columns are tied only while *no* row yet separates them; descending only ever *adds* separating levels ("cuts"), so a tie can only ever *split*, never heal. That is monotonicity (Corollary 7.8) made visible: Σ can shed permutations as Q grows, but it can never acquire one.

Remarks

1. **Where each hypothesis is used.** The no-repeats condition $\vec{v}_j, \vec{u}_j \in s(n)$ is what makes $coh(\vec{v}_j)_m$ the indicator of a genuine prefix *set*, so that prefixes nest and the threshold counts in Lemma 7.4 equal the prefix sizes. Everything else (the filter, the two orderings, the cleaning) is shown to be inert. The result holds for all $k \ge 1$ and $n \ge 2$.

2. **What does the real work.** Two readings of one idea – "a snapshot remembers how it was built". Read shallowly (the largest entry of each row) it gives the position, which neutralises *clean*. Read fully (the whole row) it gives the slot-set, which neutralises $p \ne q$. Monotonicity then turns "each snapshot is alignable" into "one permutation aligns them all".

3. **Computational corroboration.** The biconditional was checked exhaustively for $n = 3, k = 2$ (all $\vec{v}, \vec{u}$, all p, q, all $\vec{f}$) and on large random samples for $(n, k) \in \{(3, 3), (4, 2), (4, 3), (5, 2)\}$, with no violation; whenever local agreement held the two orderings were found to be in step ($\mathcal{R}_{\vec{v}} = \mathcal{R}_{\vec{u}}$), and the recovery of Lemma 7.4 was verified exhaustively for $2 \le n \le 6$.

A Source-code Correspondence

Files

File	Role
`C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`	Algorithm 2 direct; `Embedder.embed_generic`; Steps A–F. Original file on which this document is based.
`C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`	Algorithm 2 EncDec-backed; `Embedder.embed_generic`; Steps A–F. Delegates Steps A–E to `EncDec.py`; adds Step F directly. Default config is bit-identical to `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py`.
`C0HomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py`	Wrapper selecting between `C0HomDeg1_simplicialComplex_embedder_2a_for_array_of_reals_as_multiset.py` and `C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`. All other repository code imports `Embedder` from this file.
`C0HomDeg1_simplicialComplex_embedder_1_for_array_of_reals_as_multiset.py`	Algorithm 1 direct; `Embedder.embed_generic`; Steps A–F
`EncDec.py`	`simplex_2_preprocess_steps` and `simplex_1_preprocess_steps`; Steps A–E only
`play_simplex_encoders_via_EncDec.py`	Command-line driver for `EncDec.py`

Notation–code correspondence (`EncDec.py`)

The central function is `barycentric_subdivide(lin_comb, ...)`, which performs one Abel-summation step. Both algorithms call it twice: once to extract the symmetric anchor(s) and once to produce the final coefficient–basis pairs. The table below maps the document's symbols to the corresponding names in `EncDec.py`.

Document symbol	`EncDec.py` name / expression	Notes
Input M ($k\times n$)	`set_array`	Rows = coordinates i; columns = vectors j
e_i^j (Eji basis matrix)	`basis_vec` inside `MonoLinComb`	$k\times n$ indicator: 1 at (i,j), 0 elsewhere
Step A: $M = \sum_{j,i} X[j][i]\, e_i^j$	`array_to_lin_comb(set_array)` → `LinComb`	Each `(coeff, basis_vec)` term is one Eji
Anchors m_i (Alg 2); global min (Alg 1)	`offset` returned by `barycentric_subdivide(..., return_offset_separately=True)`	k per-row offsets (Alg 2); one global offset (Alg 1)
$\delta_{(s)}$, $V_{(s)}$	`.coeffs[s]`, `.basis_vecs[s]` of `lin_comb_1_first_diffs`	After 1st `barycentric_subdivide`; sorted descending
$\Delta_{(s)}$, $L_{(s)}$	`.coeffs[s]`, `.basis_vecs[s]` of `lin_comb_2_second_diffs`	2nd call with `preserve_scale=False`
α_s, $\hat{L}_{(s)}$	same	2nd call with `preserve_scale=True` (default); `coeffs[s]` $= (s{+}1)\Delta_{(s)}$, `basis_vecs[s]` $= L_{(s)}/(s{+}1)$
$\widetilde{L}_{(s)}$ (canonical)	`tools.sort_np_array_rows_lexicographically(basis_vec)`	Step E; sorts columns of each basis matrix
Step F hash	`Eji_LinComb.hash_to_point_in_unit_hypercube`	In `Eji_LinComb.py`; not yet in `EncDec.py`

The `preserve_scale` flag controls normalisation of prefix sums: `True` gives $\hat{L}_{(s)} = L_{(s)}/(s{+}1)$ with $\alpha_s = (s{+}1)\Delta_{(s)}$; `False` gives raw $L_{(s)}$ with $\Delta_{(s)}$. The second `barycentric_subdivide` call uses `preserve_scale=True` by default, matching the $\alpha_s/\hat{L}_{(s)}$ notation used throughout this document.

Differences between the two implementations

- **Step F coverage.** `EncDec.py` covers only Steps A–E. The `C0HomDeg1` files complete the embedding by hashing each canonical basis matrix to a point in the unit hypercube via `Eji_LinComb.hash_to_point_in_unit_hypercube` and forming $\sum_s \alpha_s p_s$. In `C0HomDeg1_simplicialComplex_embedder_2b_for_array_of_reals_as_multiset.py`, Step F is implemented directly in `_hash_basis_vec_to_point`, which reconstructs the integer count matrix from `EncDec.py`'s Fraction-valued basis vectors and then applies the same MD5→seed→RNG hash as `Eji_LinComb`.
- **Arithmetic.** `EncDec.py` uses `fractions.Fraction` for coefficients when `preserve_scale=True`, giving exact rational arithmetic. The `C0HomDeg1` files use `numpy` integers throughout.
- **Internal representation.** The `C0HomDeg1` files use the `Eji`, `Eji_LinComb`, and `Maximal_Simplex_Vertex` classes from their own modules. `EncDec.py` uses plain `numpy` arrays as basis vectors inside `MonoLinComb`/`LinComb`.

B Index of notation and terminology (injectivity proof)

A quick lookup for where each symbol or term is first defined. "p." is a page number, "§" a (sub)section, and "eq." a numbered equation.

$s(n)$ repeat-free length-$(n{-}1)$ sequences from $[1,n]$ (leave one symbol out, order the rest) p. 14